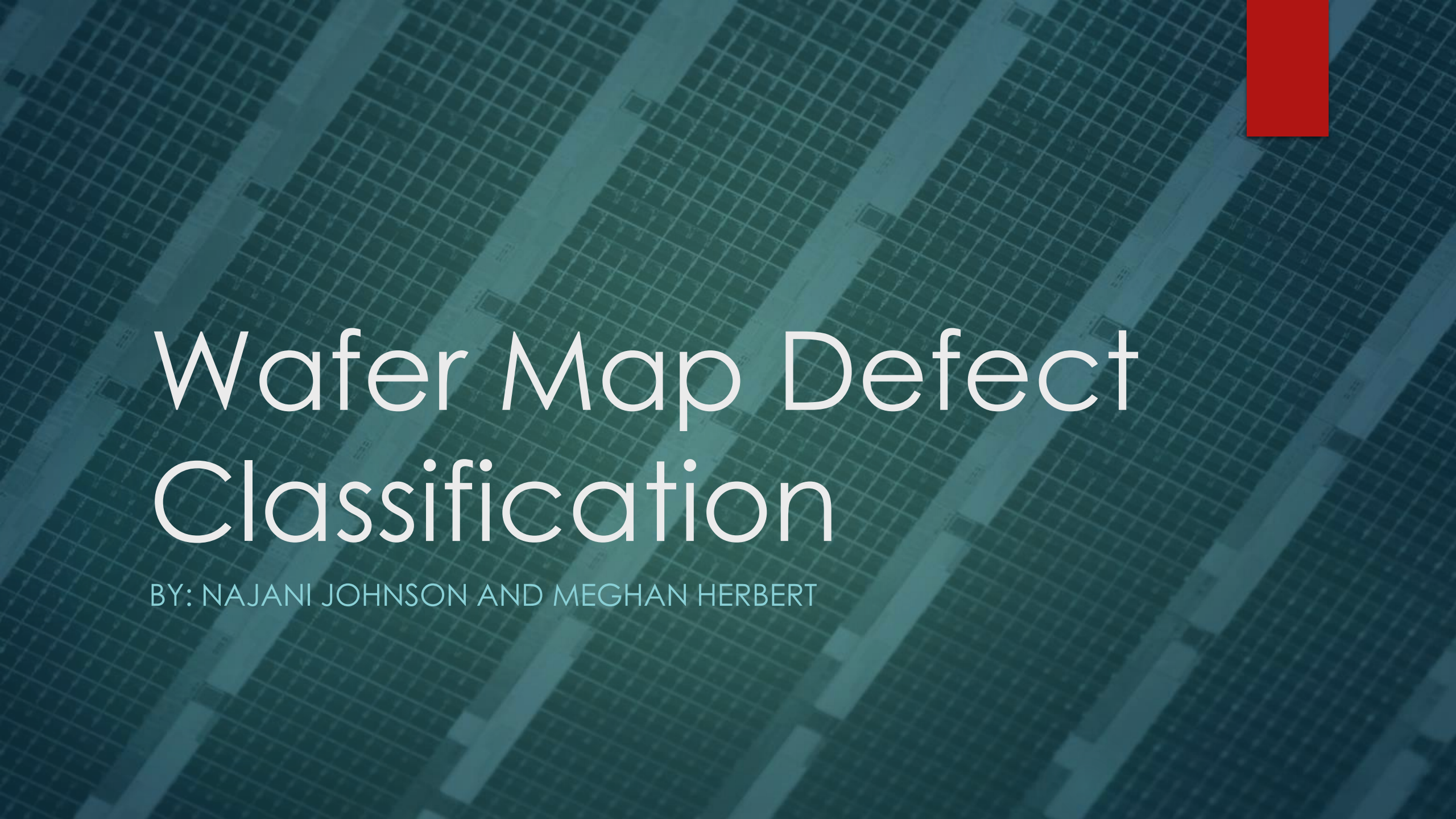




Wafer Map Defect Classification

BY: NAJANI JOHNSON AND MEGHAN HERBERT

Why Detect Wafer Defects?



Semiconductor wafers contain thousands of dies



Defects reduce manufacturing yield



Manual inspection is slow and expensive

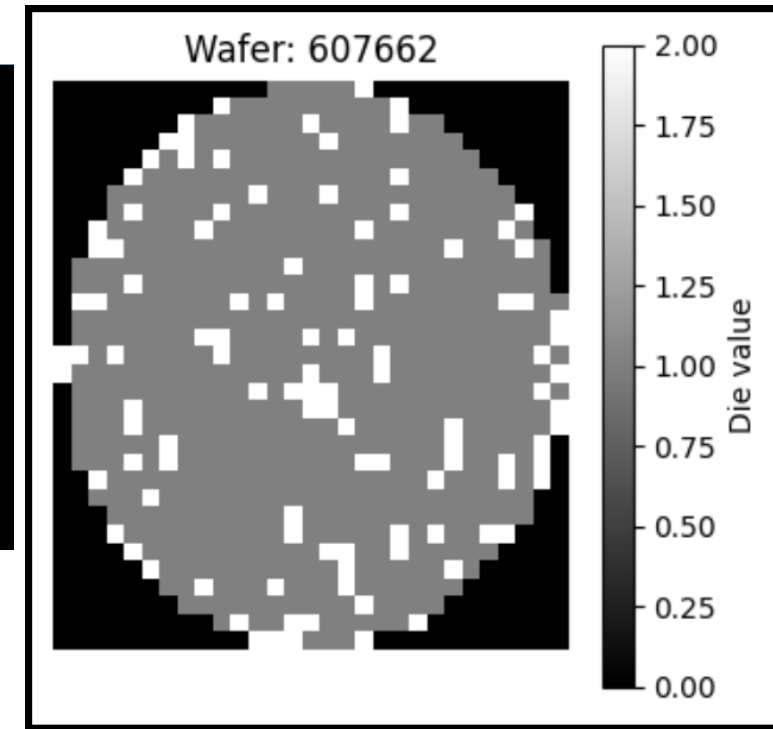


Machine learning can automate defect classification

Dataset Overview

- ▶ 811,000 wafer maps
- ▶ each wafer is a **2D grid**
- ▶ values represent die status

```
Die value meanings:  
0 = background (outside wafer)  
1 = normal functioning die  
2 = defective die  
  
Numerical wafer data:  
[[0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 2 0 0 0 0 0 0 0 0 0 0 0]  
[0 0 0 0 0 0 0 0 0 2 1 1 1 1 1 1 1 1 2 0 0 0 0 0 0 0 0 0]  
[0 0 0 0 0 0 0 2 1 1 1 1 1 1 2 1 1 1 1 2 1 1 0 0 0 0 0 0]  
[0 0 0 0 0 0 2 2 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 0 0 0 0 0]  
[0 0 0 0 0 2 1 2 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0]  
[0 0 0 0 2 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 0 0 0 0]  
[0 0 0 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0]  
[0 0 0 1 2 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 0 0]  
[0 0 2 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 2 1 0]  
[0 0 2 2 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 2 1 0]  
[0 1 1 1 1 1 1 1 1 1 1 1 1 2 1 1 1 1 1 1 1 1 1 1 1 1 1 0]
```



```
import pandas as pd

df = pd.read_csv("wm811K_csv_export.csv")

print(df.head())
print(df.info())
```

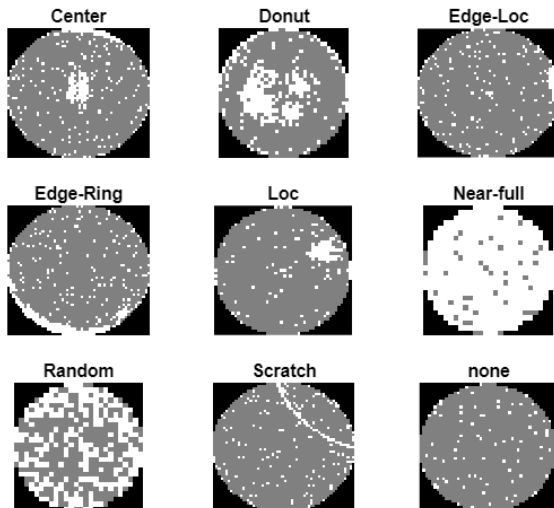
✓ 8.5s

```
print(df["failureType"].value_counts())
```

✓ 0.0s

Python

```
failureType
none      147431
Edge-Ring   9680
Edge-Loc    5189
Center     4294
Loc        3593
Scratch    1193
Random      866
Donut       555
Near-full   149
Name: count, dtype: int64
```



Importing & Preprocessing the Data

Model Implementation

Random Forest

- ▶ baseline traditional ML model
- ▶ operates on flattened wafer maps

CNN

- ▶ treats wafer maps as images
- ▶ detects spatial defect patterns

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, n_jobs=-1)

rf.fit(X_train_flat, y_train)

pred = rf.predict(X_test_flat)
```

✓ 4.6s

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

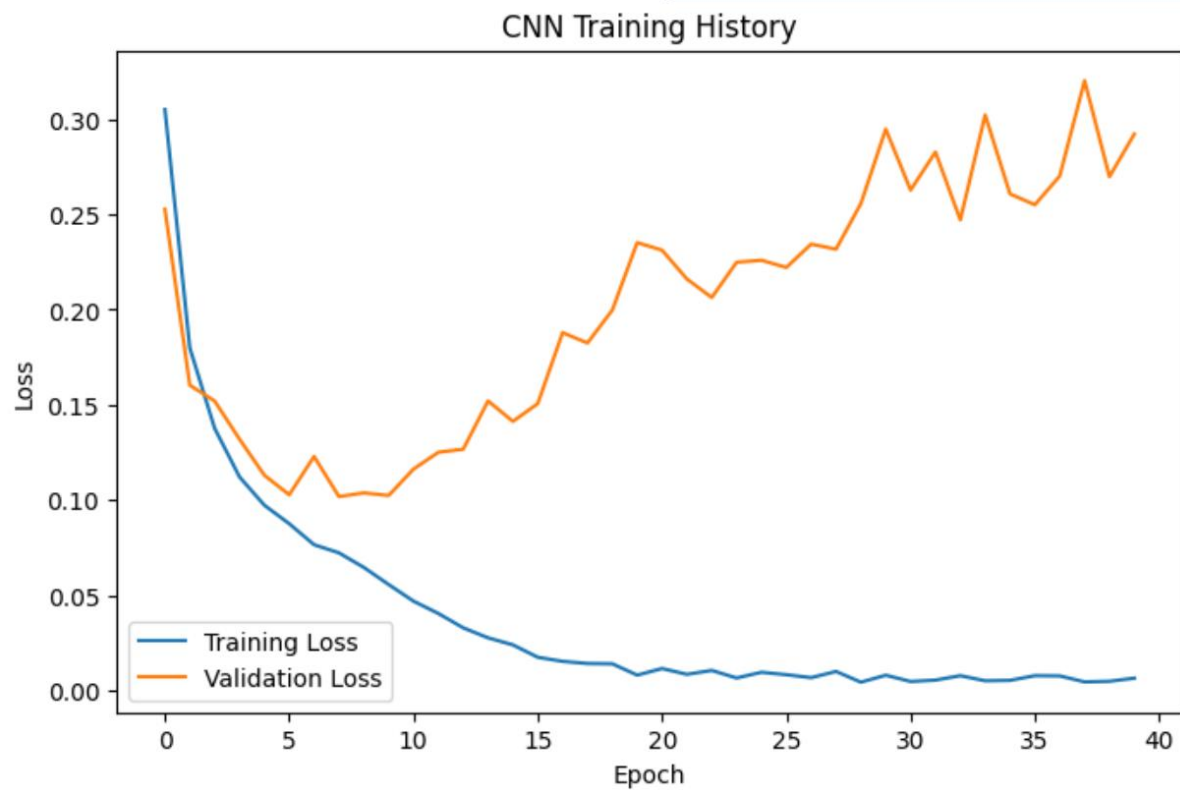
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(26,26,1)),
    MaxPooling2D(2,2),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D(2,2),

    Flatten(),
    Dense(128, activation='relu'),
    Dense(len(np.unique(y_encoded)), activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

✓ 4.5s



CNN Training Results

Training History Observation

Training Behavior

Training loss decreases steadily ($0.29 \rightarrow 0.05$).

Validation loss initially decreases but then rises ($0.21 \rightarrow 0.51$).

Divergence between curves appears around **epoch 3–5**.

Model Interpretation

This pattern indicates **overfitting**.

The CNN begins learning patterns specific to the training wafers rather than general defect structures.

Best generalization occurs **early in training**.

Possible Causes

Class imbalance in the dataset (some defect types dominate).

Limited variability in wafer map patterns.

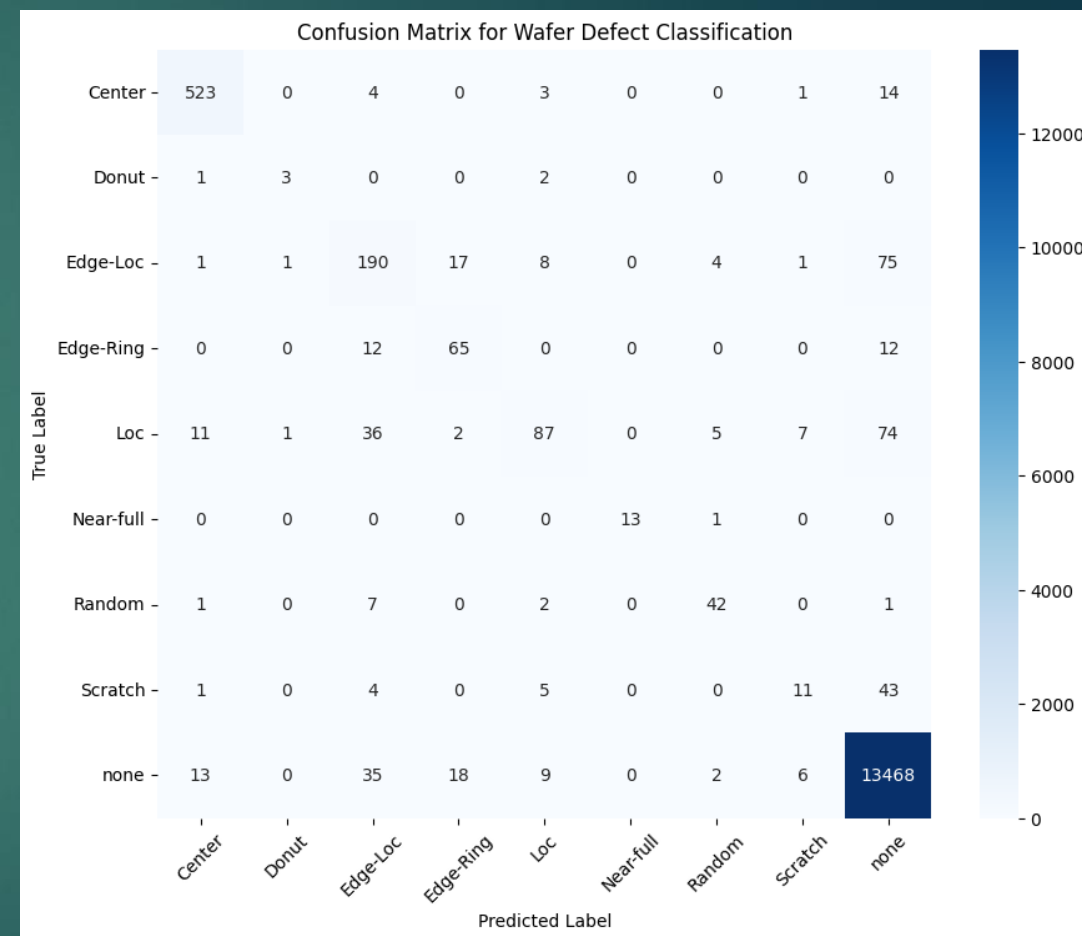
CNN architecture may be too complex relative to the signal in the data.

Key Takeaway

- ▶ The CNN learns training patterns successfully, but validation performance suggests **early overfitting**, indicating the need for **regularization or early stopping**.
- ▶ Training loss decreased steadily.
- ▶ Validation loss increased after approximately **5 epochs**.
- ▶ This suggests the CNN began **overfitting early in training**.

Random Forest Model Evaluation

	precision	recall	f1-score	support
0	0.99	0.92	0.95	545
1	1.00	0.17	0.29	6
2	0.69	0.33	0.44	297
3	0.95	0.43	0.59	89
4	0.93	0.11	0.20	223
5	1.00	0.86	0.92	14
6	0.89	0.32	0.47	53
7	1.00	0.11	0.20	64
8	0.96	1.00	0.98	13551
accuracy			0.96	14842
macro avg	0.93	0.47	0.56	14842
weighted avg	0.96	0.96	0.95	14842



Training History Observation

Training Behavior

The Random Forest model achieved **96% overall accuracy** on the test dataset.

High precision and recall were observed for **dominant classes**, particularly the **“none” category**, which contains the majority of samples.

The confusion matrix shows strong diagonal values for major classes, indicating many correct predictions.

Model Interpretation

The model performs well on **frequent defect patterns**, correctly identifying classes such as **Center defects and Near-full defects**.

However, performance varies significantly across defect types, with several classes showing **low recall and F1-scores**.

This indicates the model is better at identifying **common patterns** than distinguishing between rarer defect structures.

Possible Causes

The dataset contains **strong class imbalance**, with the **“none” class dominating the data distribution**.

Some defect patterns have **very few training examples**, making them harder for the model to learn reliably.

Certain defect patterns may have **similar spatial structures**, leading to misclassification between related defect types.

Next Steps

1

Improve CNN Model

- Reduce overfitting via early stopping by reducing number of training epochs
- Add regularization methods such as dropout layers

2

Address Class Imbalance

- Use class weighting during training
- Oversample rare defect cases

3

Expand Model Evaluation